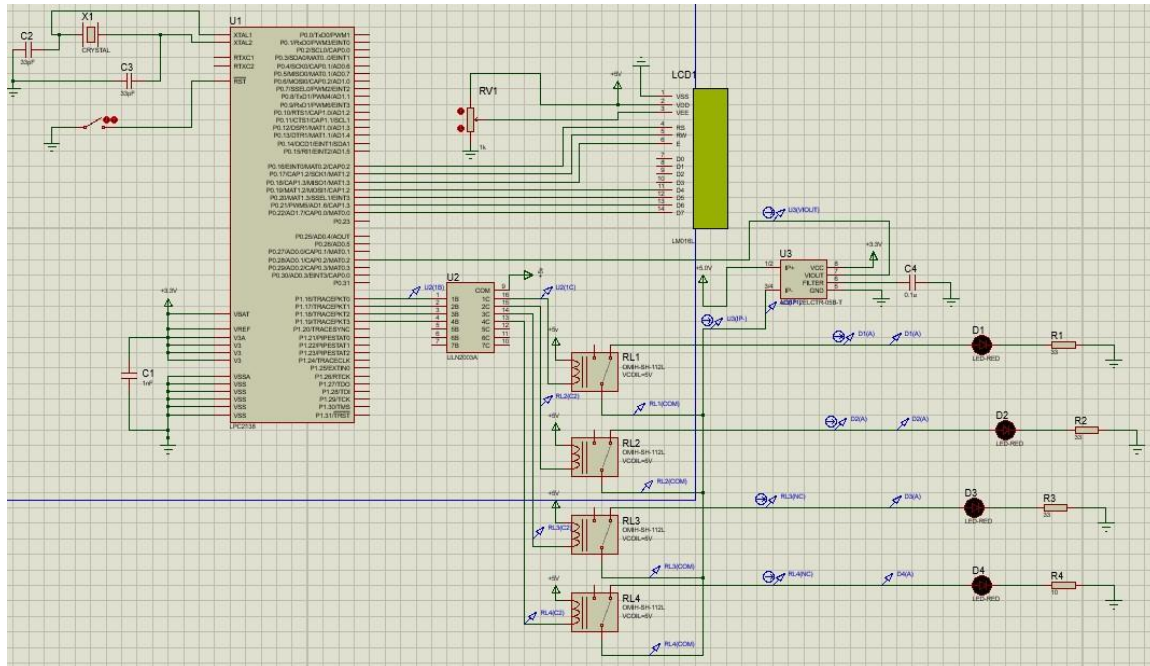


TITLE OF THE SYSTEM DESIGN PROJECT:

Energy adaptive load management using LPC2148 for low power applications

CIRCUIT DIAGRAM:



THEORY AND WORKING PRINCIPLE:

The designed circuit is a microcontroller-based(LPC2148) load monitoring and control system. It measures an analog input signal, processes it using a microcontroller, and controls multiple electrical loads through relays based on predefined threshold levels. The system also displays real-time information on a 16×2 LCD.

The operation of the system is based on three main concepts:

- **Analog-to-Digital Conversion (ADC):**

The input to the system is an analog voltage obtained from current sensor. Since microcontrollers operate on digital data, this analog signal must be converted into a digital value. The ADC inside the LPC213x microcontroller converts the input voltage into a 10-bit digital value ranging from 0 to 1023 using the relation:

$$\text{Digital Value} = (V_{in}/V_{ref}) \times 1023$$

- **Embedded Processing:**

The microcontroller reads the ADC value and compares it with predefined threshold levels. Based on this comparison, it decides which loads should remain ON or OFF.

This forms a simple control system, where:

Input → Analog voltage

Controller → Microcontroller program(Threshold Set)

Output → Relay-controlled loads

- **Relay Operation:**

Based on the value of the ADC of the Output voltage from the current sensor. Relays are being operated. Thresholds have been set to prevent loading. Relays have been set in a way based on prioritizing the loads. Load 1 has the highest priority, in homes whatever might be the load we want the refrigerator to be turned ON. In a similar fashion, the prototype has been developed. Here, the loading is defined in a way such that voltage is fixed and the amount of Current must be less than some threshold to prevent loading.(To prevent Overheating, Component Failure -LED, Transistors and so on)

Working Principle:

The working of the circuit can be explained step-by-step with respect to the program:

- **Step 1: System Initialization**

At the beginning, the microcontroller initializes all peripherals:

- LCD is configured for display
- ADC is configured to read analog input from pin P0.28
- Relay pins (P1.16–P1.19) are set as outputs

The LCD Initially displays system messages such as initialization status.

- **Step 2: Initial Relay Condition**

All relays are initially set to a default condition (normally closed or OFF state), ensuring that all loads are activated before measurement(For a Short Period).

- **Step 3: ADC Measurement**

The microcontroller reads the analog voltage using the ADC:

adc = ADC_Read();

The analog input is converted into a digital value between 0 and 1023.

- **Step 4: Decision Making:**

The Below values are defined based on successive experimentation, Although the Current drawn from the source is high for low resistances the output voltage of current sensor is differentially low. That's why the threshold values are defined closely. The ADC value is compared with predefined thresholds:

- $ADC < 520 \rightarrow$ Low load condition
- $520 - 535 \rightarrow$ Medium load condition
- $535 - 545 \rightarrow$ High load condition
- $> 545 \rightarrow$ Very high load condition

- **Step 5: Relay Control**

Based on the condition, relays are controlled as follows:

- Low load → All relays ON
- Medium load → One load OFF
- High load → Two loads OFF
- Very high load → Only one load remains ON

This process is similar to automatic load shedding, where loads are reduced as demand increases.

- **Step 6: Display Output** The LCD displays:

- The ADC value
- The corresponding load status

Example outputs:

- “L1L2L3L4 ON”
- “L4 OFF”
- “L3L4 OFF”

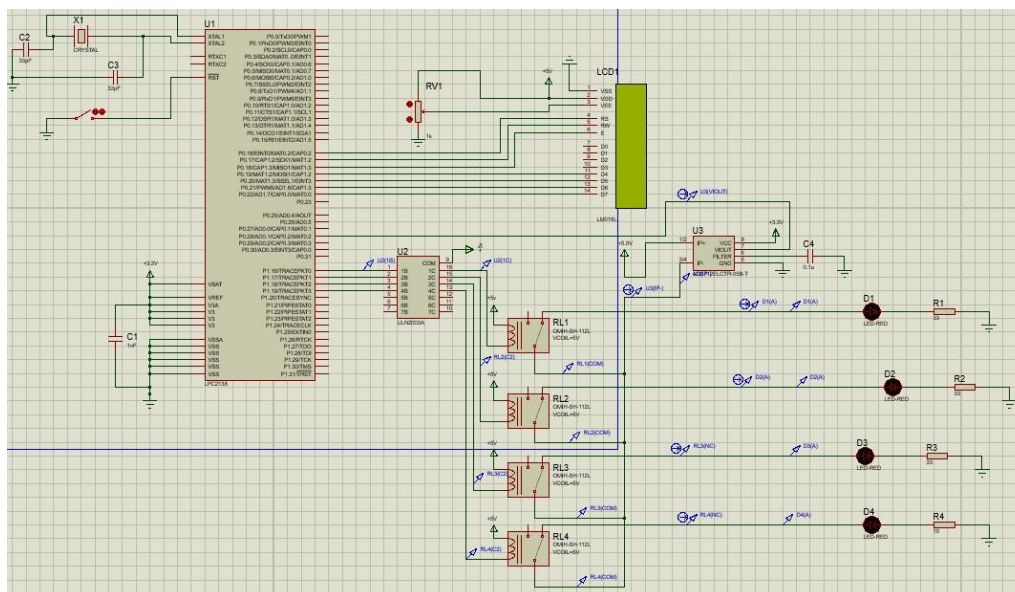
- **Step 7:**

Based on the output of the system , the user needs to decide whether the output meets the requirement or not, if not a better load must be selected for working.

TOOLS & TECHNOLOGY USED:

Proteus 8 professional , Keil μ vision 4 , Flash Magic

SCREENSHOT OF THE SIMULATOR (PROTEUS) BASED CIRCUIT IMPLEMENTATION:

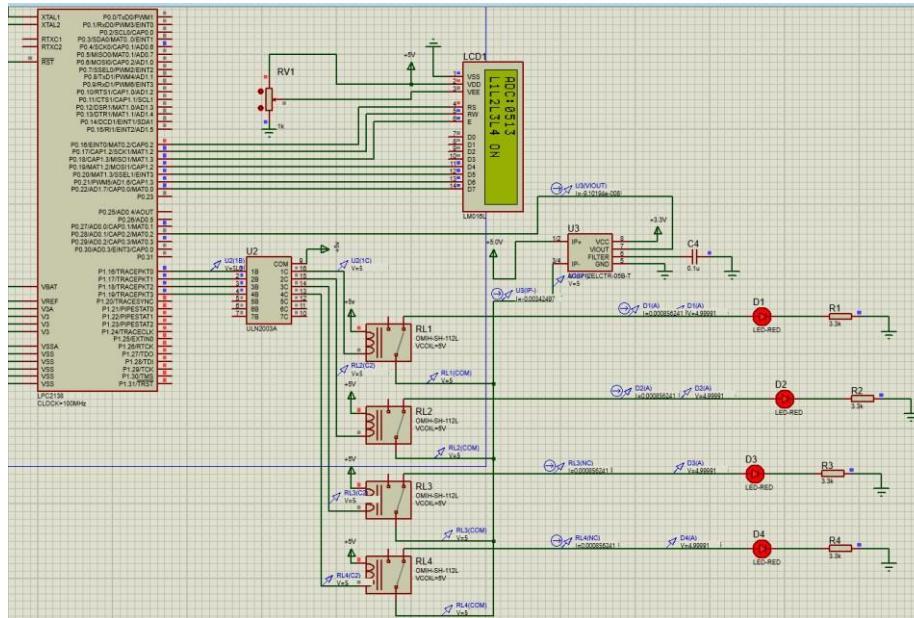


SCREENSHOT OF THE OUTPUT OBTAINED FROM THE SIMULATOR:

Case I: All loads are ON (Less Current Loading for a Fixed Voltage)

Example: $R1=3.3k\Omega$, $R2=3.3k\Omega$, $R3=3.3k\Omega$, $R4=3.3k\Omega$

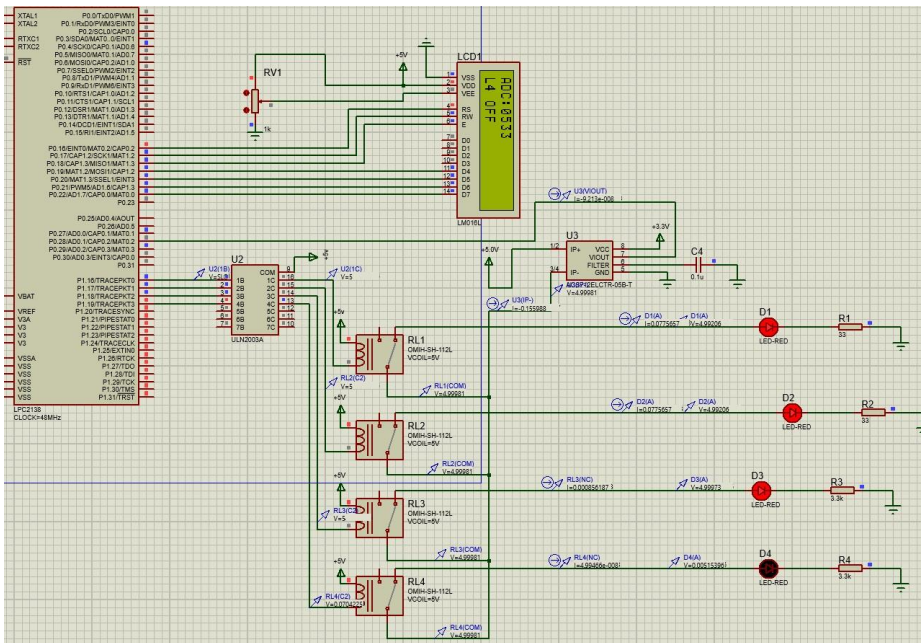
ADC Value: 513



Case II: Only Load 4 is OFF

Example: $R1=33\Omega$, $R2=33\Omega$, $R3=3.3k\Omega$, $R4=3.3k\Omega$

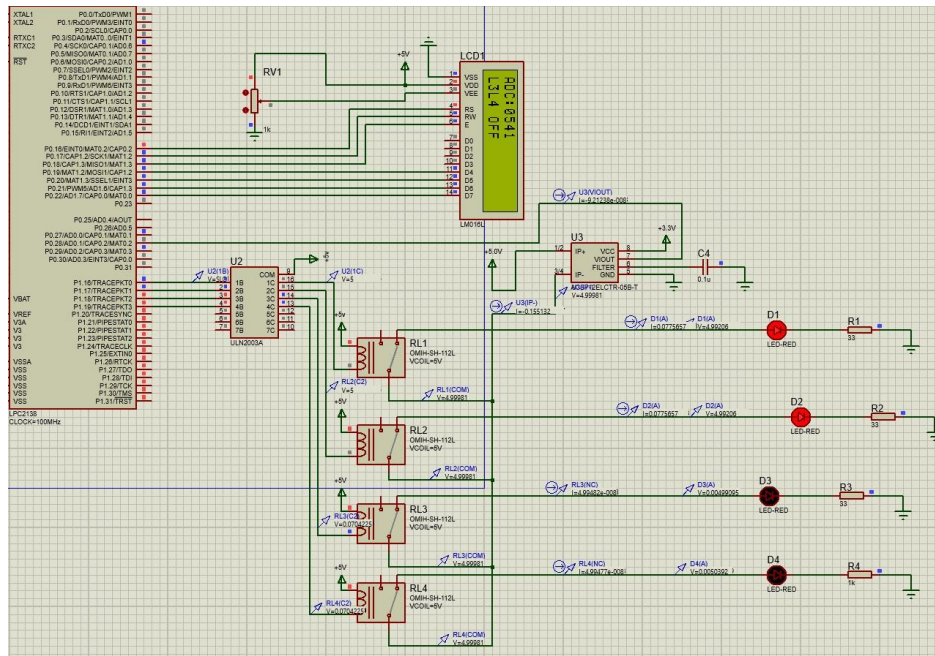
ADC Value: 533



Case III: Only Load 1, Load 2 are ON

Example: $R_1 = 33\Omega$, $R_2 = 33\Omega$, $R_3 = 33\Omega$, $R_4 = 1k\Omega$

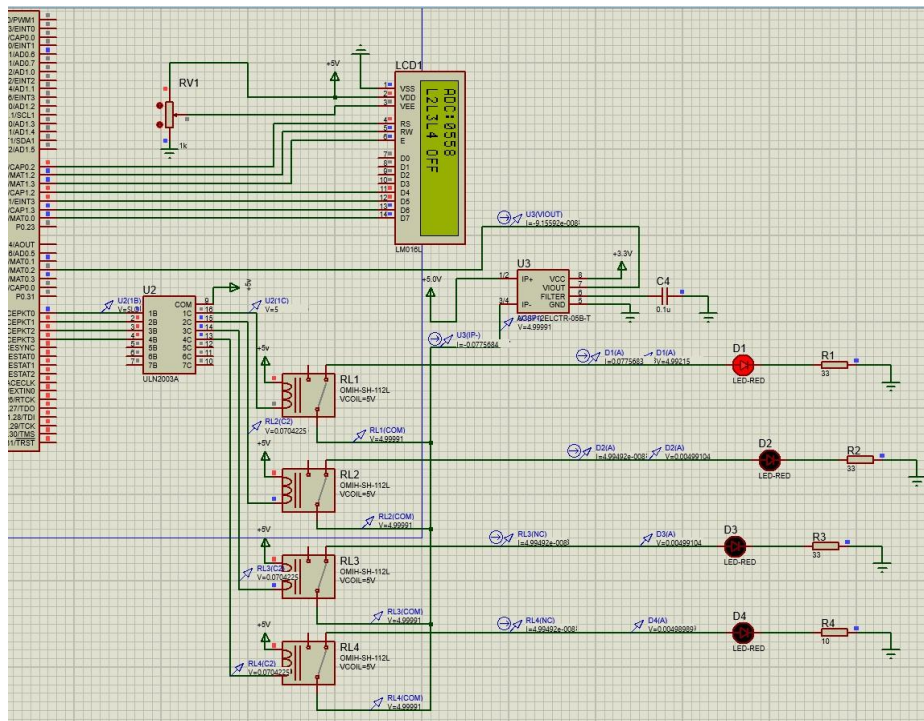
ADC Value: 541



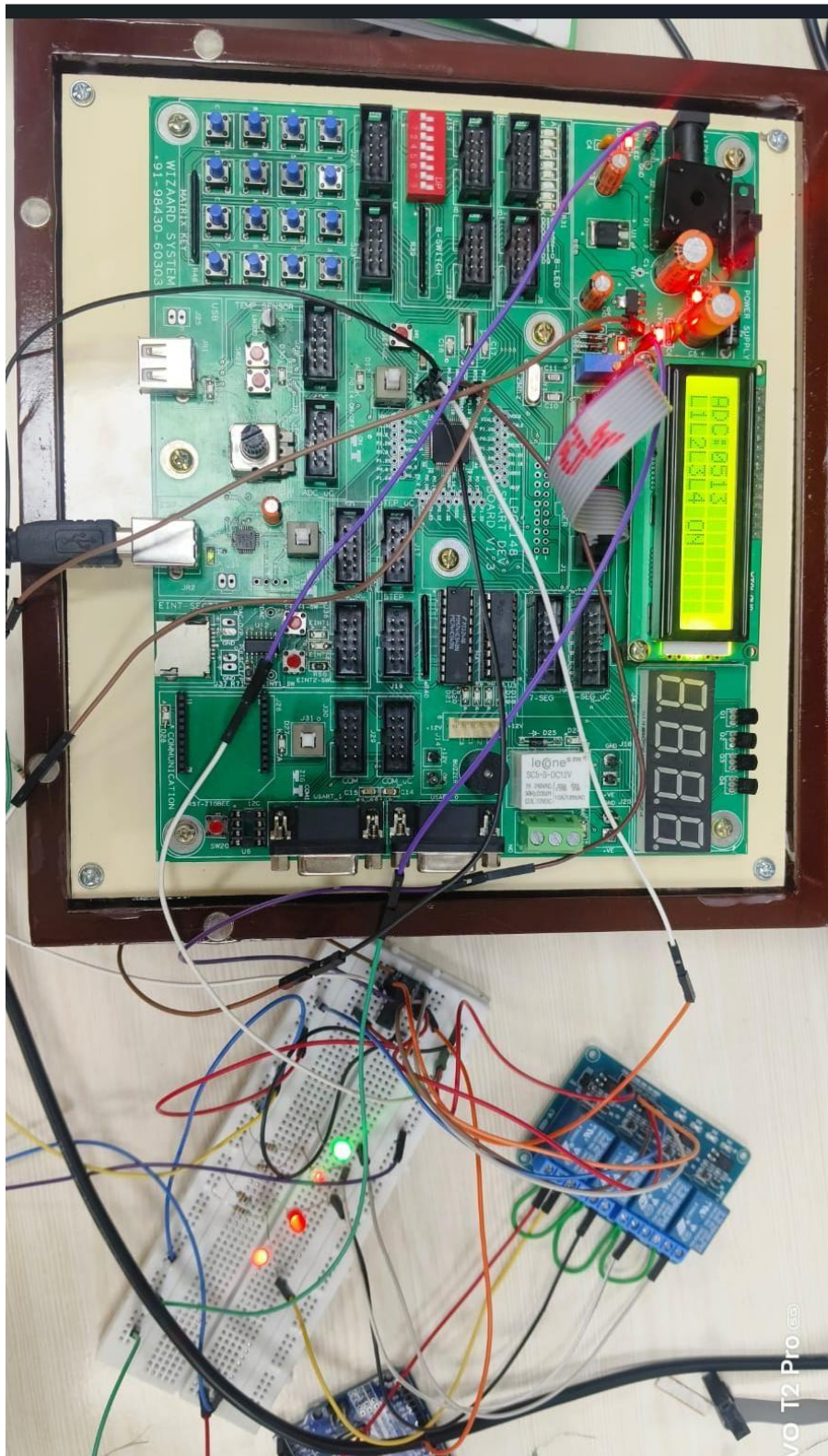
Case IV: Only Load 1 is ON (More Current Loading for a Fixed Voltage)

Example: $R_1 = 33\Omega$, $R_2 = 33\Omega$, $R_3 = 33\Omega$, $R_4 = 10\Omega$

ADC Value: 558



SCREENSHOT OF THE CIRCUIT BASED ON HARDWARE KIT:

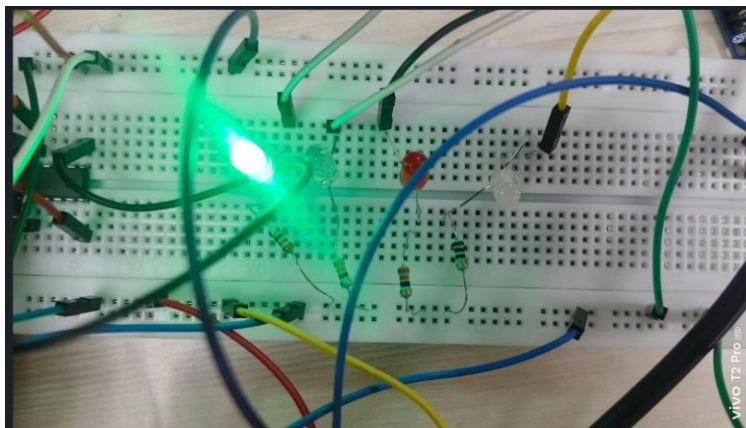
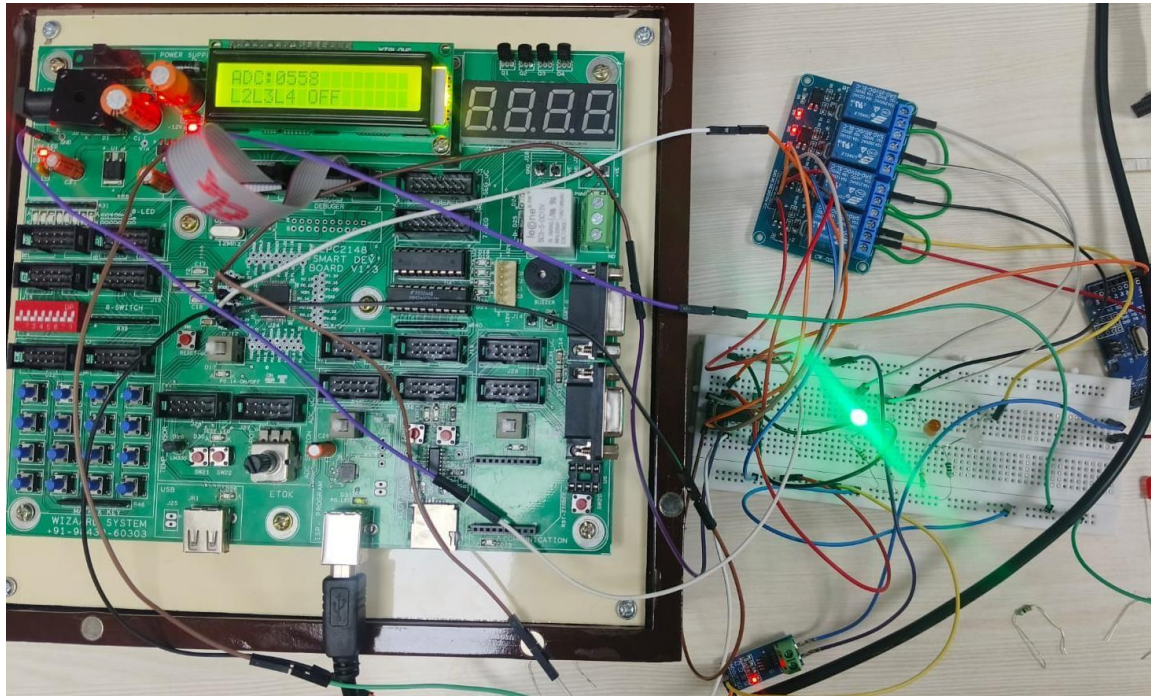


SCREENSHOT OF THE OUTPUT OBTAINED FROM THE HARDWARE FOR THE CIRCUIT:

Case I: Only Load 1 is ON (More Current Loading for a Fixed Voltage)

Example: $R_1 = 33\Omega$, $R_2 = 33\Omega$, $R_3 = 33\Omega$, $R_4 = 10\Omega$

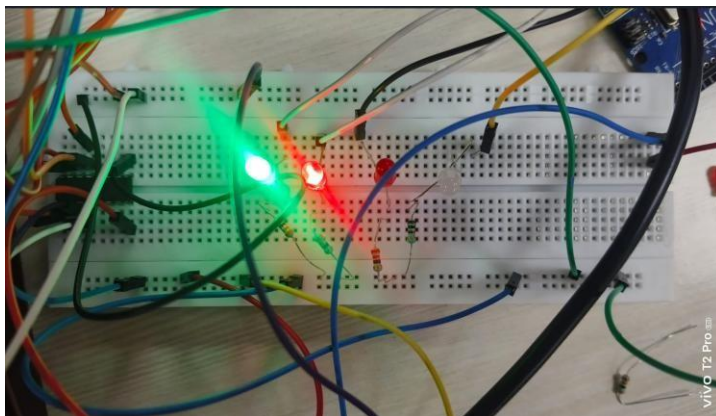
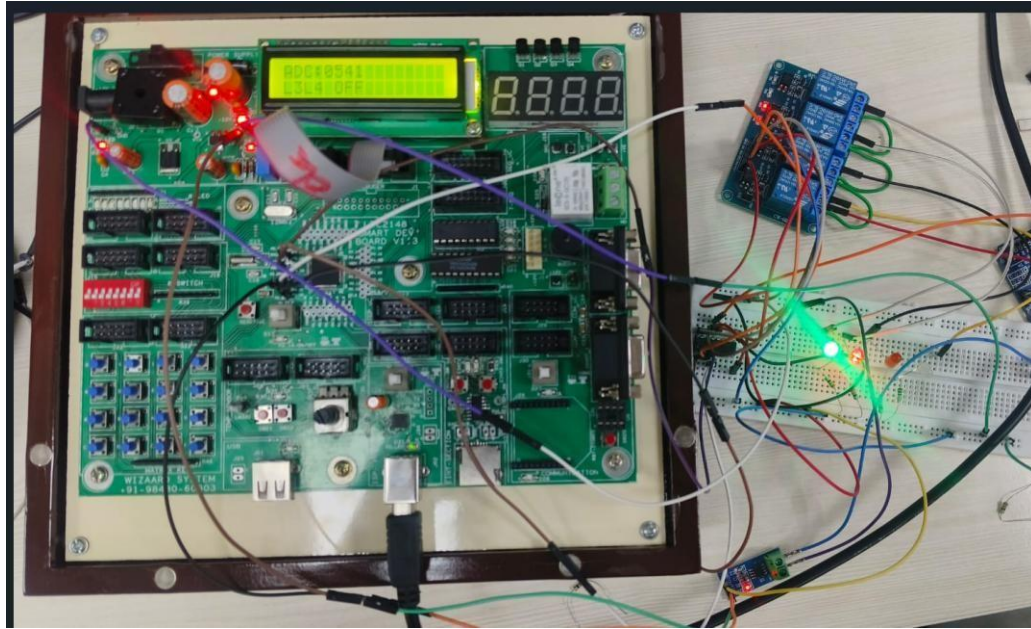
ADC Value: 558



Case II: Only Load 1, Load 2 are ON

Example: $R_1 = 33\Omega$, $R_2 = 33\Omega$, $R_3 = 33\Omega$, $R_4 = 1k\Omega$

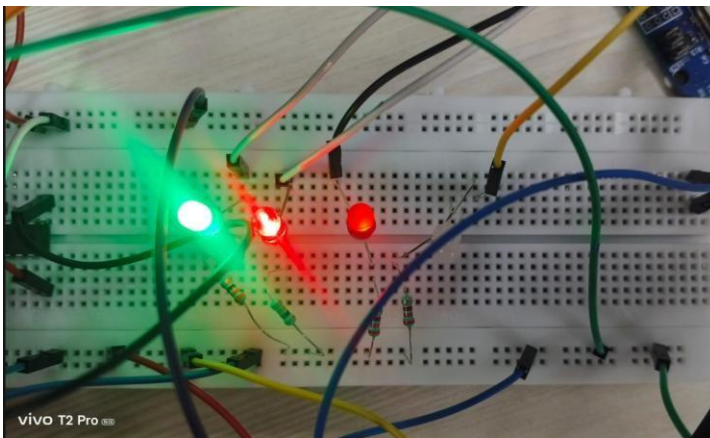
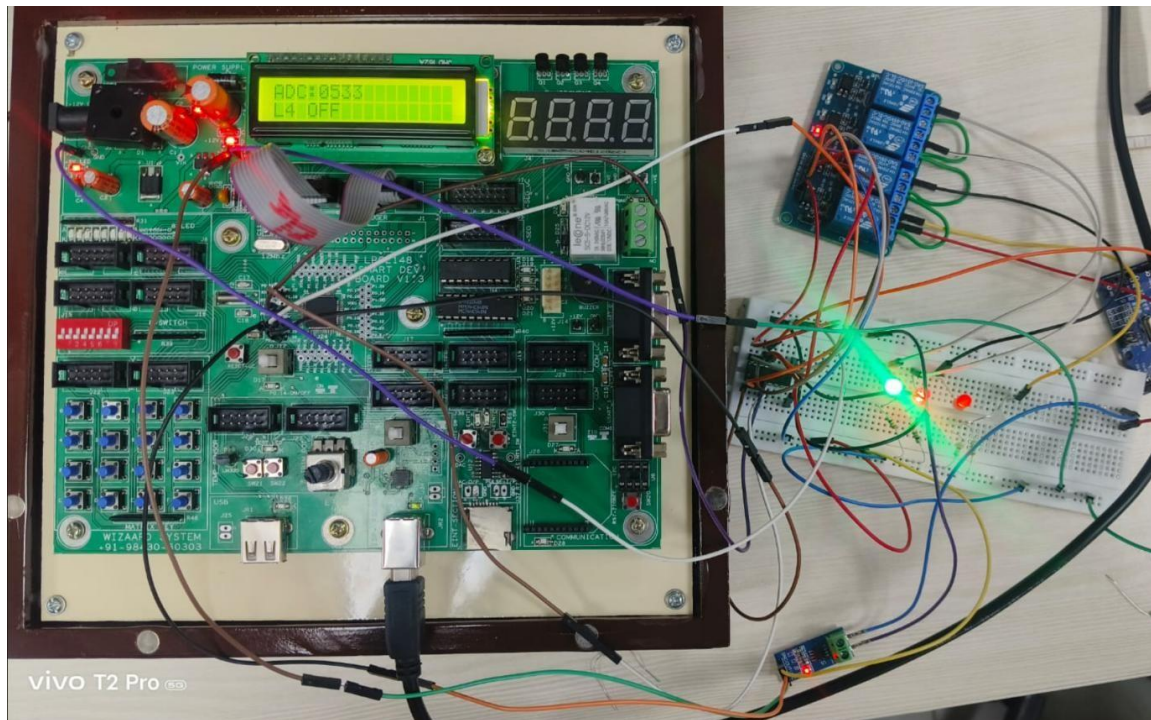
ADC Value: 541



Case III: Only Load 4 is OFF

Example: $R_1=33\Omega$, $R_2=33\Omega$, $R_3=3.3k\Omega$, $R_4=3.3k\Omega$

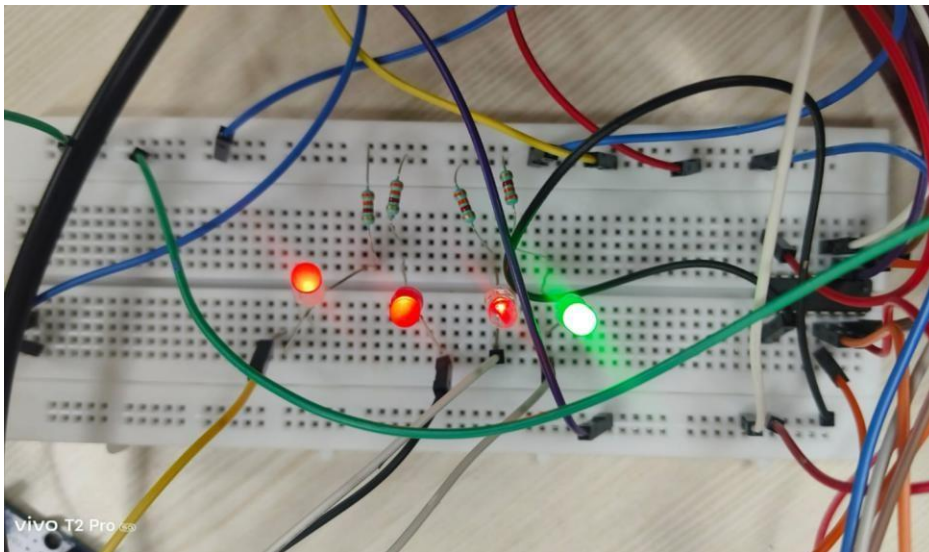
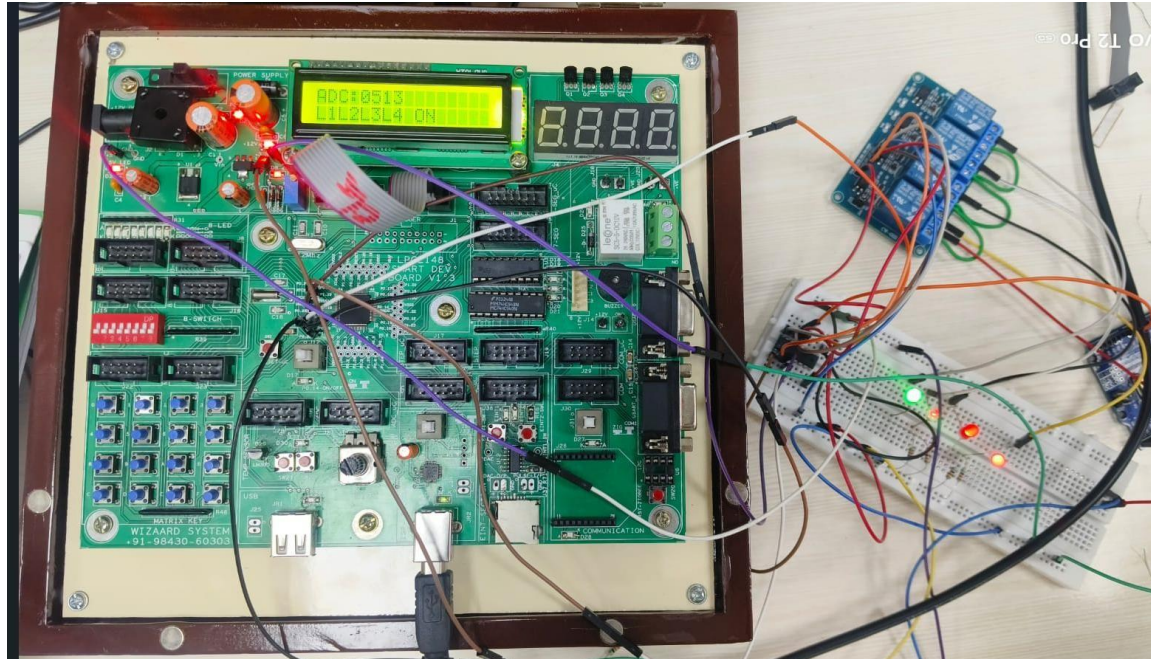
ADC Value: 533



Case IV: All loads are ON (Less Current Loading for a Fixed Voltage)

Example: $R_1=3.3k\Omega$, $R_2=3.3k\Omega$, $R_3=3.3k\Omega$, $R_4=3.3k\Omega$

ADC Value: 513



SINGLE VIDEO LINK FOR THE WORKING MODEL OF THE SOFTWARE CIRCUIT AND HARDWARE CIRCUIT:

[Batch 03 MCI laboratory SLD Video.mp4](#)

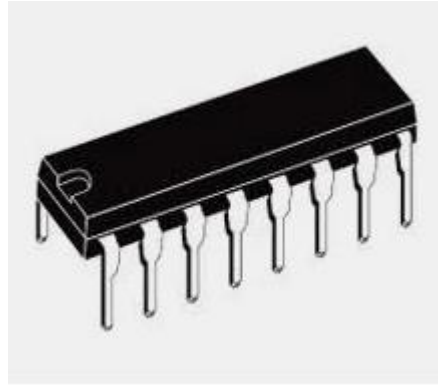
Component Level Explanation:

Relay and Relay driver(ULN2003A):

Relay(5V Nominal Voltage)



ULN2003A(Relay Driver)



The circuit uses an electromagnetic relay (OMIH-5H, 5V) in combination with the ULN2003A driver IC to enable a low-power microcontroller to control higher-current loads safely and effectively. The working of both components is closely linked, as the relay cannot be driven directly by the microcontroller and depends on the driver stage for proper operation.

The ULN2003A is a Darlington transistor array designed to act as a current amplifier and switch. When the microcontroller outputs a logic HIGH signal to one of the ULN2003A input pins, the corresponding Darlington pair inside the IC turns ON. This creates a low-resistance path between the output pin of the ULN2003A and ground. As a result, current is allowed to flow from the power supply through the relay coil and into the ULN2003A, which sinks the current to ground. In this way, the ULN2003A acts as an interface that boosts the current from the microcontroller to a level sufficient to drive the relay coil.

When the microcontroller output is LOW, the Darlington transistor inside the ULN2003A remains OFF, and no current flows through the relay coil. This keeps the relay in its default state. One important feature of the ULN2003A is the presence of internal flyback diodes connected to a common pin. These diodes protect the circuit from high voltage spikes generated when the relay coil is suddenly de-energized. Without this protection, the back electromotive force (EMF) produced by the collapsing magnetic field in the coil could damage the driver or microcontroller.

The OMIH-5H relay itself operates on the principle of electromagnetism. It consists of a coil, an armature, and a set of contacts labelled Common (COM), Normally Open (NO), and Normally Closed (NC). When current flows through the coil, a magnetic field is generated, which attracts the armature and causes the internal switch to change position. In this energized state, the COM terminal connects to the NO terminal, allowing current to pass through the external load connected to these contacts. At the same time, the connection between COM and NC is broken.

When the current through the coil is interrupted, the magnetic field collapses, and a spring mechanism returns the armature to its original position. In this de-energized state, the COM terminal reconnects to the NC terminal, and the NO contact becomes open again. This switching action allows the relay to control circuits that may operate at higher voltages or currents than the control circuitry.

In the overall operation of the circuit, the microcontroller determines when a relay should be activated. It sends a control signal to the ULN2003A, which then drives the relay coil by providing the required current path to ground. The relay responds by switching its contacts, thereby controlling the connected load. This arrangement ensures electrical isolation between the control side and the load side, improves reliability, and protects sensitive electronic components from electrical stress.

Current Sensor:



The current sensor is used to monitor how much current is flowing through the load, rather than controlling it. It is placed in series with the load path, which means all the current going to the loads must pass through the sensor. This allows it to “observe” the flow without interrupting normal operation.

Inside the sensor, the flowing current creates a physical effect (typically magnetic or resistive), which is then converted into a small electrical signal. This signal is conditioned internally and provided as a smooth analog voltage output. The important idea is that as current increases, the sensor’s output shifts proportionally, and when current decreases, the output shifts back. The output is centered around a reference level when no current is flowing, and it moves above or below that level depending on the direction and magnitude of current.

The microcontroller reads this output and interprets it to understand the load condition—whether the system is lightly loaded, normally operating, or overloaded.

It is better practice to fix the current sensor’s Vcc as 3.3 Volt, such that the output voltage of the current Sensor doesn’t exceed 3.3 volt because ADC of LPC2148 requires a maximum voltage of 3.3 volt. If it exceeds 3.3 volt then.. An error in reading is produced.

The loads (LED branches through relays) are effectively connected in a parallel configuration. In a parallel system, all loads share the same supply voltage. This means the voltage across each branch remains constant regardless of how many loads are connected or switched ON.

What changes is the total current drawn from the source. Each load draws its own share of current depending on its resistance or power requirement. When more loads are turned ON (more relays activated), additional current paths are created. As a result, the total current increases because the source is now supplying multiple branches simultaneously.

This is why we talk about current loading rather than voltage loading. The voltage is fixed by the power supply and remains stable across all branches. However, the current demand grows as more loads are connected, which places a greater “load” on the system

The current sensor continuously feeds information to the microcontroller. Based on this:

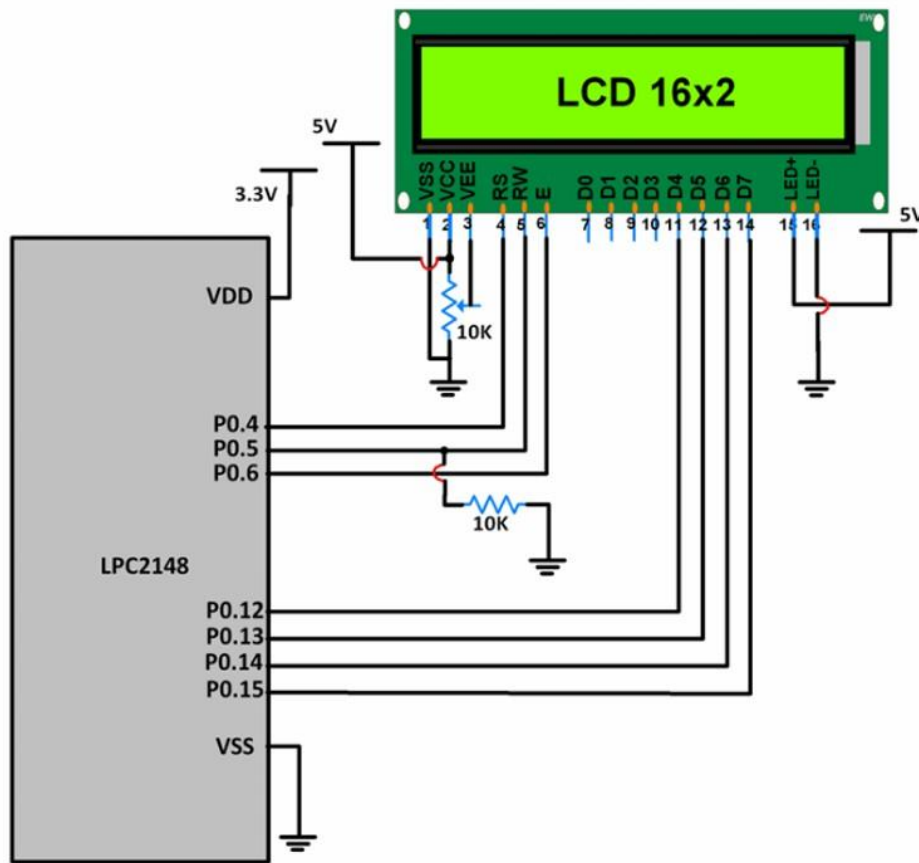
- If current is within safe range → system continues normal operation
- If current increases beyond a threshold → system identifies heavy load
- If current becomes too high → microcontroller can take action such as:
 - Turning OFF some relays
 - Triggering an alert
 - Displaying warning on LCD

This makes the system intelligent, as it does not blindly switch loads but monitors their effect on the power system.

LCD:

The LCD in this circuit acts as a visual output interface, allowing the user to see real-time information about the system instead of relying only on internal processing. It is connected to the microcontroller, which continuously sends data to be displayed based on the circuit’s operation and sensor readings.

In this project, the LCD primarily displays the current status of the system, such as which loads (relays) are ON or OFF and the amount of current being drawn. The microcontroller reads the signal from the current sensor, interprets the loading condition, and then updates the LCD accordingly. For example, when fewer loads are active, the display may indicate a normal or low-load condition, while activating more loads increases current and the LCD reflects a higher load status.



The LCD also helps in decision visibility. When the system detects an overload condition based on current sensing, the microcontroller can take action (like turning OFF some relays), and the LCD shows this change instantly. This makes it easier to understand how the system is responding without needing external measurement tools.

Another important role of the LCD is user interaction and debugging. During implementation and testing, it provides immediate feedback about whether the circuit is working correctly, whether sensors are giving proper readings, and whether relays are switching as expected. This reduces the difficulty of troubleshooting.

Overall, the LCD converts internal electrical behaviour into readable information, making the system user-friendly, transparent, and easier to monitor in real time.

CONCLUSION:

The developed system successfully demonstrates an efficient and intelligent method for controlling and monitoring electrical loads using a microcontroller, relay driver, and

current sensing unit. The relays provide reliable switching of multiple loads while maintaining electrical isolation between the control and power sections. The ULN2003A driver ensures that the microcontroller can safely handle higher current requirements without damage, making the overall design robust and practical.

A key feature of the project is the integration of the current sensor, which enables real-time monitoring of load conditions. Since the loads are connected in parallel, the voltage across each load remains constant, while the total current drawn varies depending on the number of active loads. By observing this current variation, the system can accurately determine the loading condition rather than relying on voltage changes, which are minimal in such configurations.

Based on the sensed current, the microcontroller can make informed decisions to maintain system safety and efficiency. This includes identifying overload conditions, controlling relays accordingly, and providing feedback through indicators or display units. Such functionality enhances the reliability of the system and prevents potential damage due to excessive current draw.

Overall, the project highlights the importance of current-based load monitoring in parallel circuits and demonstrates a practical approach to smart load control. The design can be further extended for real-world applications such as home automation, industrial load management, and energy-efficient systems.

PROBLEMS FACED DURING THE IMPLEMENTATION:

- Power supply instability – when multiple relays turn ON, the total current increases and can cause voltage drops or reset the microcontroller
- Relay switching noise – electrical spikes generated during ON/OFF switching can disturb the microcontroller and sensor signals
- Current sensor inaccuracies – noise and environmental factors can lead to incorrect current readings and wrong decisions
- Wiring and connection errors – complex parallel connections increase chances of loose or incorrect wiring
- Grounding issues – improper common ground can cause unstable operation and faulty signal readings
- Overloading in parallel circuit – adding more loads increases total current and may exceed safe limits
- Heat generation – continuous high current through relays and driver IC can cause overheating and reduce component life

PROBLEMS RESOLVED USING SOLUTIONS:

- Power supply instability - Use a regulated, high-current power supply In real setups, when multiple relays switch ON together, the supply may dip and reset the microcontroller. This is resolved by using a power supply with sufficient current rating and adding filter capacitors near the MCU and relay section to maintain stable voltage.
- Relay switching noise - Add proper decoupling and grounding
During practical operation, relay switching introduces noise that can disturb signals. This is reduced by ensuring a common ground, using short wiring, and placing decoupling capacitors across supply lines to absorb spikes.
- Current sensor fluctuations - Calibrate and filter readings
In actual conditions, sensor output may not be stable. This is handled by calibrating the sensor for no-load and full-load conditions, and applying software filtering (like averaging) in the microcontroller to get stable readings.
- Wiring errors - Use proper layout and labeling
While assembling the circuit, incorrect or loose connections are common. This is avoided by using clear wiring layouts, proper labeling, and checking continuity before powering the system.
- Grounding issues - Maintain a single common ground reference
In practical circuits, improper grounding leads to erratic behavior. Ensuring all components (MCU, ULN2003A, sensor, relays) share a common ground stabilizes the system.
- Overloading in parallel loads - Set current limits in code
When too many loads are connected, the total current increases beyond safe levels. This is resolved by programming the microcontroller to monitor current and turn OFF some relays when a threshold is exceeded.
- Heat generation - Provide heat management and proper rating
Continuous operation with high current can heat components. This is handled by selecting properly rated relays and driver ICs, ensuring ventilation, and avoiding operation near maximum limits.

CODE:

```
#include <LPC214x.h>
```

```
#define RS (1U << 16)
#define RW (1U << 17)
#define EN (1U << 18)
#define D4 (1U << 19)
```

```

#define D5 (1U << 20)
#define D6 (1U << 21)
#define D7 (1U << 22)

#define RELAY1 (1U << 16)
#define RELAY2 (1U << 17)
#define RELAY3 (1U << 18)
#define RELAY4 (1U << 19)

void delay_us(unsigned int us)
{
    unsigned int i, j;
    for(i=0;i<us;i++)
        for(j=0;j<10;j++);
}

void delay_ms(unsigned int ms)
{
    unsigned int i, j;
    for(i=0;i<ms;i++)
        for(j=0;j<6000;j++);
}

void lcd_send_nibble(unsigned char nibble)
{
    IOCLR0 = D4|D5|D6|D7;

    if(nibble & 1) IOSET0 = D4;
    if(nibble & 2) IOSET0 = D5;
    if(nibble & 4) IOSET0 = D6;
    if(nibble & 8) IOSET0 = D7;

    IOSET0      =      EN;
    delay_us(50);
    IOCLR0 = EN;
}

void lcd_cmd(unsigned char cmd)
{
    IOCLR0 = RS;
    IOCLR0 = RW;

    lcd_send_nibble(cmd>>4);
    lcd_send_nibble(cmd);

    delay_ms(2);
}

```

```

}

void lcd_data(unsigned char data)
{
    IOSET0 = RS;
    IOCLR0 = RW;

    lcd_send_nibble(data>>4);
    lcd_send_nibble(data);

    delay_ms(2);
}

void lcd_init(void)
{
    PINSEL1 &= ~0x00003FFF;

    IODIR0 |= RS|RW|EN|D4|D5|D6|D7;
    IOCLR0 = RS|RW|EN|D4|D5|D6|D7;

    delay_ms(20);

    lcd_send_nibble(0x03); delay_ms(5);
    lcd_send_nibble(0x03); delay_ms(5);
    lcd_send_nibble(0x03); delay_ms(5);
    lcd_send_nibble(0x02);

    lcd_cmd(0x28);
    lcd_cmd(0x0C);
    lcd_cmd(0x06); lcd_cmd(0x01);
}

void lcd_string(char *s)
{
    while(*s) lcd_data(*s++);
}

void lcd_gotoxy(unsigned char pos)
{
    lcd_cmd(pos);
}

void lcd_clear(void)

```

```

{
    lcd_cmd(0x01);
    delay_ms(5);
}

void lcd_print_num(unsigned int num)
{
    lcd_data((num/1000)+'0');
    lcd_data(((num/100)%10)+'0');
    lcd_data(((num/10)%10)+'0');
    lcd_data((num%10)+'0');
}

void lcd_print_status(unsigned char level)
{
    lcd_gotoxy(0xC0);

    switch(level)
    {
        case 0: lcd_string("L1L2L3L4 ON "); break;
        case 1: lcd_string("L4 OFF "); break;
        case 2: lcd_string("L3L4 OFF "); break;
        case 3: lcd_string("L2L3L4 OFF "); break;
        default: lcd_string("UNKNOWN ");
    }
}

void ADC_Init(void)
{
    VPBDIV = 0;

    // P0.28 -> AD0.1
    PINSEL1 &= ~(3U << 24);
    PINSEL1 |= (1U << 24);

    // Select channel 1
    AD0CR = (1<<1) | (15<<8) | (1<<21);
}

unsigned int ADC_Read(void)
{
    unsigned int result;

    AD0CR |= (1<<24);
}

```

```

    while(!(AD0GDR & (1U<<31)));

    result = AD0GDR;

    AD0CR &= ~(7U<<24);

    return (result >> 6) & 0x3FF;
}

void relays_init(void)
{
    PINSEL2 = 0;
    IO1DIR |= RELAY1|RELAY2|RELAY3|RELAY4;

    IO1SET = RELAY1|RELAY2|RELAY3|RELAY4;
}

void set_relays(unsigned char r1,unsigned char r2,unsigned char r3,unsigned char r4)
{
    if(r1) IO1CLR = RELAY1; else IO1SET = RELAY1;
    if(r2) IO1CLR = RELAY2; else IO1SET = RELAY2;    if(r3)
    IO1CLR = RELAY3; else IO1SET = RELAY3;    if(r4)
    IO1CLR = RELAY4; else IO1SET = RELAY4;
}

int main(void)
{
    unsigned int
    adc;
    unsigned char level;

    lcd_init();
    ADC_Init();
    relays_init();

    lcd_string("Initializing...");
    delay_ms(1500);
    lcd_clear();

    lcd_clear();
    lcd_string("GPIO P1.16-19");
    lcd_gotoxy(0xC0);
    lcd_string("LOADS ON");

```

```

set_relays(1,1,1,1);
delay_ms(500);
    lcd_clear();
    lcd_string("relays in NC"); // loads OFF before ADC

    lcd_clear();
    lcd_string("Computing ADC");
    delay_ms(1000);

    adc      =      ADC_Read();
    lcd_clear();

    if(adc < 520)
    {
        set_relays(1,1,1,1); // low load: all ON
        level = 0;
    }
    else if(adc < 535 && adc>520)
    {
        set_relays(1,1,1,0); // medium load: L4 OFF
        level = 1;
    }
    else if(adc < 545 && adc>535)
    {
        set_relays(1,1,0,0); // high load: L3,L4 OFF
        level = 2;
    }
    else
    {
        set_relays(1,0,0,0); // very high load: only L1 ON
        level = 3;
    }

    while(1)
    {
        lcd_gotoxy(0x80);
        lcd_string("ADC:");
        lcd_print_num(adc);
        lcd_print_status(level);
        delay_ms(300);
    }
}

```

REFERENCES:

- Working of Relay and Its Applications <https://robocraze.com/blogs/post/what-is-relay-working-applications-types?srsltid=AfmBOopWwk9yO5mJvxBgt6ANDcCEyfxPM2aG50qgOETaXHkpTYaKxNN>
- ULN2003A Datasheet – Texas Instruments
<https://www.ti.com/lit/ds/symlink/uln2003a.pdf>
<https://www.electronicclinic.com/arduino-uln2003-driver-to-control-a-relay/>
- ACS712 Current Sensor Overview <https://www.allegromicro.com/-/media/files/datasheets/acs712-datasheet.ashx>
- Parallel Circuits / Loading <https://www.allaboutcircuits.com/textbook/direct-current/chpt-5/simple-parallelcircuits/>
- LPC2148 User Manual:
[TB2_Smart LPC2148 User Manual.pdf](#)
- Flash Magic Tool Documentation <https://www.flashmagictool.com/>
- Keil uVision Documentation <https://www.keil.com/>
- Research papers and online tutorials